Name: Indrajeet Singh

T11, 09

Experiment-09

1. Aim

To install Docker on the system and execute basic Docker commands to verify successful installation and container functionality.

---

2. Objectives

The objectives of this experiment are:

1. **To install Docker Engine on the system.**
2. **To execute essential Docker commands.**
3. **To understand how Docker images and containers work.**
4. **To verify installation using test images.**
5. **To observe and analyze Docker outputs.**

---

3. Theory

3.1 What is Docker?

Docker is an open-source platform designed to automate the deployment, scaling, and management of applications using containerization. Containers package an application along with all its dependencies, ensuring it runs the same way regardless of the environment.

3.2 Why Docker?

- **Lightweight**
- **Faster than virtual machines**

- **Portable**
- **Consistent environment**
- **Easy to deploy**

3.3 Docker Architecture

Docker architecture has the following components:

1. **Docker Client** – sends commands (`docker run`, `docker pull`).
2. **Docker Daemon** – runs in background, manages containers.
3. **Images** – read-only templates used to create containers.
4. **Containers** – running instances of images.
5. **Docker Registry** – stores images (e.g., Docker Hub).

---

4. Installation Steps (With Screenshots)

Step 1: Verify Docker Installation

```
C:\Users\L15>
C:\Users\L15>docker --version
Docker version 29.2.1, build a5c7197

C:\Users\L15>docker info
Client:
 Version:    29.2.1
 Context:    desktop-linux
 Debug Mode: false
 Plugins:
  agent: create or run AI agents (Docker Inc.)
    Version:  v1.27.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-agent.exe
  ai: Docker AI Agent - Ask Gordon (Docker Inc.)
    Version:  v1.18.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-ai.exe
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.31.1-desktop.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-buildx.exe
  compose: Docker Compose (Docker Inc.)
    Version:  v5.0.2
    Path:     C:\Program Files\Docker\cli-plugins\docker-compose.exe
  debug: Get a shell into any image or container (Docker Inc.)
    Version:  0.0.47
    Path:     C:\Program Files\Docker\cli-plugins\docker-debug.exe
  desktop: Docker Desktop commands (Docker Inc.)
    Version:  v0.3.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-desktop.exe
  extension: Manages Docker extensions (Docker Inc.)
    Version:  v0.2.31
    Path:     C:\Program Files\Docker\cli-plugins\docker-extension.exe
  init: Creates Docker-related starter files for your project (Docker Inc.)
    Version:  v1.4.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-init.exe
  mcp: Docker MCP Plugin (Docker Inc.)
    Version:  v0.40.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-mcp.exe
  model: Docker Model Runner (Docker Inc.)
    Version:  v1.1.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-model.exe
  offload: Docker Offload (Docker Inc.)
    Version:  v0.5.56
    Path:     C:\Program Files\Docker\cli-plugins\docker-offload.exe
  pass: Docker Pass Secrets Manager Plugin (beta) (Docker Inc.)
    Version:  v0.0.24
    Path:     C:\Program Files\Docker\cli-plugins\docker-pass.exe
  sandbox: Docker Sandbox (Docker Inc.)
    Version:  v0.12.0
```

Step 2: Check Docker Information

```
  Profile: builtin
 cgroupns
Kernel Version: 6.6.87.2-microsoft-standard-WSL2
Operating System: Docker Desktop
OSType: linux
Architecture: x86_64
CPUs: 20
Total Memory: 15.5GiB
Name: docker-desktop
ID: 17c859d5-1964-42ba-9f87-c9101ed97da6
Docker Root Dir: /var/lib/docker
Debug Mode: false
HTTP Proxy: http.docker.internal:3128
HTTPS Proxy: http.docker.internal:3128
No Proxy: hubproxy.docker.internal
Labels:
 com.docker.desktop.address=npipe://\\.\pipe\docker_cli
Experimental: false
Insecure Registries:
 hubproxy.docker.internal:5555
 ::1/128
 127.0.0.0/8
Live Restore Enabled: false
Firewall Backend: iptables
```

Step 3: Run a Test Container

```
C:\Users\L15>docker pull hello-world
Using default tag: latest
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
ea52d2000f90: Download complete
Digest: sha256:ef54e839ef541993b4e87f25e752f7cf4238fa55f017957c2eb44077083d7a6a
Status: Downloaded newer image for hello-world:latest
docker.io/library/hello-world:latest
```

Step 4: View Pulled Images

```
docker.io/library/hello-world:latest

C:\Users\L15>docker images

IMAGE                              ID             DISK USAGE   CONTENT SIZE   EXTRA
docker/welcome-to-docker:latest    c4d56c24da4f       22.2MB         6.03MB    U
hello-world:latest                 ef54e839ef54       25.9kB         9.52kB
```

Step 5: View Running Containers

```
C:\Users\L15>docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED        STATUS        PORTS                                      NAMES
e71abffcc66f   docker/welcome-to-docker:latest   "/docker-entrypoint.…"   4 minutes ago   Up 7 minutes   0.0.0.0:8088->80/tcp, [::]:8088->80/tcp   welcome-to-docker
```

6. Output Observation

- Docker version was successfully displayed.
- Docker info showed system configuration.
- “hello-world” image was pulled and executed.
- Images list included newly downloaded images.
- Running container status was verified via `docker ps`.

7. Conclusion

In this experiment, Docker was successfully installed and tested using basic commands.
We learned how to:

- verify installation
- run images
- list containers and images
- understand basic Docker workflow

- The experiment demonstrates how Docker simplifies application deployment using lightweight containers.